

JavaScript Arrays — Mutation & Copying

Array নিয়ে কাজ করার সময় সবচেয়ে important conceptগুলোর একটি হলো:

এই operation কি original array পরিবর্তন করছে, নাকি নতুন array তৈরি করছে?

এই distinction না বুঝলে JavaScript এবং React—দুই জায়গাতেই অনেক bug হতে পারে।

এই document-এ আমরা শিখব:

- `push()`
- `pop()`
- `shift()`
- `unshift()`
- `splice()`
- `slice()`
- Mutating vs non-mutating methods
- Return values
- Insert / delete / replace
- Array copying
- Shallow copy
- React-এ immutable array updates

1. Mutable vs Non-Mutating

প্রথমে সবচেয়ে important distinction:

Mutating methods

Original array পরিবর্তন করে:

```
push
pop
shift
unshift
splice
```

Non-mutating methods

Original array পরিবর্তন না করে নতুন result দেয়:

```
slice
```

আর `map()`, `filter()`, `flat()` ইত্যাদিও সাধারণত নতুন array তৈরি করে—এগুলো পরের iteration doc-এ আসবে।

2. `push()`

`push()` array-এর শেষে এক বা একাধিক element যোগ করে।

```
const fruits = ["Apple", "Banana"];

fruits.push("Mango");

console.log(fruits);
```

Result:

```
["Apple", "Banana", "Mango"]
```

3. `push()` Original Array Mutate করে

```
const fruits = ["Apple", "Banana"];

const result = fruits.push("Mango");

console.log(fruits);
// ["Apple", "Banana", "Mango"]
```

অর্থাৎ original array পরিবর্তিত হয়েছে।

4. `push()` কী Return করে?

`push()` নতুন array return করে না।

এটা **updated array**-এর নতুন **length** return করে।

```
const fruits = ["Apple", "Banana"];

const result = fruits.push("Mango");

console.log(result);

// 3
```

কারণ updated array-এর length **3**।

5. Multiple Elements with **push()**

একসাথে একাধিক element:

```
const fruits = ["Apple"];

fruits.push("Banana", "Mango", "Orange");

console.log(fruits);
```

Result:

```
["Apple", "Banana", "Mango", "Orange"]
```

6. **pop()**

pop() array-এর শেষ element remove করে।

```
const fruits = ["Apple", "Banana", "Mango"];

fruits.pop();

console.log(fruits);
```

Result:

```
["Apple", "Banana"]
```

7. `pop()` কী Return করে?

`pop()` যে element remove করেছে, সেটাই return করে।

```
const fruits = ["Apple", "Banana", "Mango"];

const removed = fruits.pop();

console.log(removed);
// "Mango"
```

এটা `push()` থেকে আলাদা।

```
push → new length
pop → removed element
```

8. `shift()`

`shift()` array-এর প্রথম element remove করে।

```
const fruits = ["Apple", "Banana", "Mango"];

fruits.shift();

console.log(fruits);
```

Result:

```
["Banana", "Mango"]
```

9. `shift()` কী Return করে?

যে element remove হয়েছে:

```
const fruits = ["Apple", "Banana", "Mango"];

const removed = fruits.shift();

console.log(removed);
// "Apple"
```

10. `unshift()`

`unshift()` array-এর শুরুতে element যোগ করে।

```
const fruits = ["Banana", "Mango"];

fruits.unshift("Apple");

console.log(fruits);
```

Result:

```
["Apple", "Banana", "Mango"]
```

11. `unshift()` কী Return করে?

Updated array-এর নতুন length:

```
const fruits = ["Banana", "Mango"];

const result = fruits.unshift("Apple");

console.log(result);
// 3
```

12. Four Basic Mutation Methods

এই চারটা খুব ভালোভাবে মনে রাখো:

Method	কাজ	কোথায়	Return
<code>push()</code>	Add	End	New length
<code>pop()</code>	Remove	End	Removed element
<code>unshift()</code>	Add	Start	New length
<code>shift()</code>	Remove	Start	Removed element

Mental model:

	START	END
	↓	↓
Add	<code>unshift()</code>	<code>push()</code>
Remove	<code>shift()</code>	<code>pop()</code>

13. `splice()`

`splice()` হলো সবচেয়ে powerful array mutation method।
এটা দিয়ে:

- element delete
- element insert
- element replace

সব করা যায়।

Basic syntax:

```
array.splice(start, deleteCount, item1, item2, ...);
```

14. splice() — Delete

```
const fruits = [  
  "Apple",  
  "Banana",  
  "Mango",  
  "Orange"  
];  
  
fruits.splice(1, 1);  
  
console.log(fruits);
```

Result:

```
["Apple", "Mango", "Orange"]
```

Explanation:

```
start = 1  
deleteCount = 1
```

অর্থাৎ index 1 থেকে 1টি element delete।

15. splice() Index

Index:

0	1	2	3
↓	↓	↓	↓
Apple	Banana	Mango	Orange

```
fruits.splice(1, 1);
```

মানে:

```
index 1 থেকে  
1 element delete
```

তাই **Banana** চলে যাবে।

16. Delete Multiple Elements

```
const fruits = [  
  "Apple",  
  "Banana",  
  "Mango",  
  "Orange",  
  "Grape"  
];  
  
fruits.splice(1, 3);  
  
console.log(fruits);
```

Result:

```
["Apple", "Grape"]
```

কারণ index **1** থেকে ৩টি:

```
Banana  
Mango  
Orange
```

remove হয়েছে।

17. **splice()** — Insert

deleteCount যদি **0** হয়, তাহলে শুধু insert করা যায়।

```
const fruits = [  
  "Apple",  
  "Mango"  
];  
  
fruits.splice(1, 0, "Banana");  
  
console.log(fruits);
```

Result:

```
["Apple", "Banana", "Mango"]
```

মানে:

```
start = 1  
deleteCount = 0  
insert = "Banana"
```

18. Insert Multiple Elements

```
const fruits = ["Apple", "Orange"];  
  
fruits.splice(  
  1,  
  0,  
  "Banana",  
  "Mango"  
);  
  
console.log(fruits);
```

Result:

```
["Apple", "Banana", "Mango", "Orange"]
```

19. splice() — Replace

একই operation দিয়ে delete এবং insert করলে replacement করা যায়।

```
const fruits = [  
  "Apple",  
  "Banana",  
  "Mango"  
];  
  
fruits.splice(1, 1, "Orange");  
  
console.log(fruits);
```

Result:

```
["Apple", "Orange", "Mango"]
```

এখানে:

```
Banana → remove  
Orange → insert
```

20. Replace Multiple Elements

```
const fruits = [
  "Apple",
  "Banana",
  "Mango",
  "Orange"
];

fruits.splice(
  1,
  2,
  "Grape",
  "Watermelon"
);

console.log(fruits);
```

Result:

```
[
  "Apple",
  "Grape",
  "Watermelon",
  "Orange"
]
```

21. `splice()` Return Value

`splice()` removed elements-এর একটি **new array** return করে।

```
const fruits = [  
  "Apple",  
  "Banana",  
  "Mango"  
];  
  
const removed = fruits.splice(1, 1);  
  
console.log(removed);  
// ["Banana"]  
  
console.log(fruits);  
// ["Apple", "Mango"]
```

থেয়াল করো:

```
splice()  
→ original array mutate করে  
→ removed elements-এর array return করে
```

22. `splice()` with Negative Start

Negative start index ব্যবহার করা যায়।

```
const fruits = [  
  "Apple",  
  "Banana",  
  "Mango",  
  "Orange"  
];  
  
fruits.splice(-1, 1);  
  
console.log(fruits);
```

Result:

```
["Apple", "Banana", "Mango"]
```

`-1` মানে last element থেকে শুরু।

23. `splice()` দিয়ে Last Element Remove

এভাবে করা যায়:

```
const fruits = ["Apple", "Banana", "Mango"];

fruits.splice(-1, 1);

console.log(fruits);
```

তবে শুধু last element remove করতে হলে:

```
fruits.pop();
```

আরও readable।

24. `slice()`

`slice()` array-এর একটি অংশ **copy** করে নতুন array দেয়।

Syntax:

```
array.slice(start, end);
```

`end` included নয়।

25. Basic `slice()`

```
const fruits = [  
  "Apple",  
  "Banana",  
  "Mango",  
  "Orange"  
];  
  
const result = fruits.slice(1, 3);  
  
console.log(result);
```

Result:

```
["Banana", "Mango"]
```

কারণ:

```
start = 1  
end = 3  
  
indexes:  
1 → Banana  
2 → Mango  
3 → excluded
```

26. `slice()` Does NOT Mutate

```
const fruits = [
  "Apple",
  "Banana",
  "Mango"
];

const result = fruits.slice(1, 2);

console.log(fruits);
// ["Apple", "Banana", "Mango"]

console.log(result);
// ["Banana"]
```

Original array unchanged!

27. `slice()` Without End

```
const fruits = [
  "Apple",
  "Banana",
  "Mango",
  "Orange"
];

const result = fruits.slice(2);

console.log(result);
```

Result:

```
["Mango", "Orange"]
```

মানে index `2` থেকে শেষ পর্যন্ত।

28. `slice()` from Beginning

```
const fruits = [
  "Apple",
  "Banana",
  "Mango"
];

const result = fruits.slice(0, 2);

console.log(result);
```

Result:

```
["Apple", "Banana"]
```

29. `slice()` with Negative Index

Negative index দিয়ে end থেকে count করা যায়।

```
const fruits = [
  "Apple",
  "Banana",
  "Mango",
  "Orange"
];

console.log(fruits.slice(-2));
```

Result:

```
["Mango", "Orange"]
```

কারণ `-2` মানে শেষ থেকে দ্বিতীয় position।

30. `slice()` দিয়ে Array Copy

কোনো argument না দিলে পুরো array-এর shallow copy পাওয়া যায়।

```
const original = [1, 2, 3];

const copy = original.slice();

console.log(copy);
// [1, 2, 3]

console.log(copy === original);
// false
```

এটা basic shallow copy technique।

31. `splice()` vs `slice()`

এটা interview এবং practical development—দুই জায়গাতেই খুব important।

Feature	<code>splice()</code>	<code>slice()</code>
Original array mutate করে?	✓ Yes	✗ No
নতুন array return করে?	Removed items-এর array	Selected items-এর array
Delete করতে পারে?	✓	✗
Insert করতে পারে?	✓	✗
Replace করতে পারে?	✓	✗
সাধারণত copy করতে পারে?	✗	✓

Mental model

```
splice
→ "Original array-এ surgery করো"

slice
→ "একটা অংশের copy দাও"
```

32. `push()` vs `unshift()`

push
→ end-এ add

unshift
→ start-এ add

Example:

```
const arr = [2, 3];

arr.push(4);
// [2, 3, 4]

arr.unshift(1);
// [1, 2, 3, 4]
```

33. `pop()` vs `shift()`

pop
→ end থেকে remove

shift
→ start থেকে remove

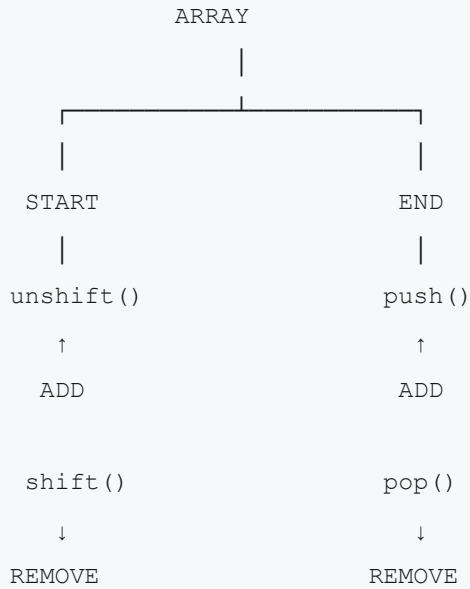
Example:

```
const arr = [1, 2, 3];

arr.pop();
// [1, 2]

arr.shift();
// [2]
```

34. Mutation Visual Map



আর:

```
splice()  
→ anywhere  
→ add / remove / replace  
→ MUTATES
```

35. Shallow Copy

Array copy করার সময় একটা important concept হলো **shallow copy**।

```
const original = [1, 2, 3];  
  
const copy = [...original];
```

এখানে outer array নতুন।

```
console.log(original === copy);  
// false
```

কিন্তু nested object থাকলে:

```
const original = [
  { name: "A" },
  { name: "B" }
];

const copy = [...original];
```

Inner objects একই reference।

36. Shallow Copy Example

```
const users = [
  { name: "A" },
  { name: "B" }
];

const copy = [...users];

copy[0].name = "C";

console.log(users[0].name);
// "C"
```

কারণ:

```
Outer array → copied
Inner objects → shared
```

37. Copying with `slice()`

```
const original = [1, 2, 3];

const copy = original.slice();

console.log(copy === original);
// false
```

এটাও shallow copy।

38. Copying with `Array.from()`

```
const original = [1, 2, 3];

const copy = Array.from(original);

console.log(copy === original);
// false
```

এটাও shallow copy।

39. Three Common Shallow Copy Techniques

```
const original = [1, 2, 3];

const copy1 = [...original];

const copy2 = original.slice();

const copy3 = Array.from(original);
```

তিনটিই নতুন outer array তৈরি করে।

40. Why Mutation Matters in React

React state-এর ক্ষেত্রে direct mutation avoid করা খুব important।

ধরো:

```
const [items, setItems] = useState([
  "Apple",
  "Banana"
]);
```

এভাবে করা উচিত নয়:

```
items.push("Mango");

setItems(items);
```

কারণ তুমি existing array mutate করেছো।

41. React-এ Immutable Add

Instead:

```
setItems([
  ...items,
  "Mango"
]);
```

এখানে নতুন array তৈরি হচ্ছে।

```
old array
  ↓
[Apple, Banana]

new array
  ↓
[Apple, Banana, Mango]
```

42. React-এ Remove

ধরো index `1` remove করতে চাও।

Direct mutation:

```
items.splice(1, 1);
```

Avoid করো।

Instead `filter()` ব্যবহার করা ভালো:

```
setItems(  
  items.filter((_, index) => index !== 1)  
);
```

`filter()` পরের doc-এ বিস্তারিত আসবে।

43. React-এ Replace an Element

ধরো index `1` update করতে হবে।

Mutation:

```
items[1] = "Orange";
```

Avoid করো।

Instead:

```
setItems(  
  items.map((item, index) =>  
    index === 1 ? "Orange" : item  
  )  
);
```

`map()` -এর বিস্তারিত পরের doc-এ থাকবে।

44. Why Immutable Updates?

React-এর state update-এর ক্ষেত্রে নতুন reference তৈরি করা useful কারণ React state-এর পরিবর্তন detect এবং render scheduling-এর ক্ষেত্রে reference identity গুরুত্বপূর্ণ।

তাই:

```
setItems(newArray);
```

যেখানে `newArray !== oldArray`, সাধারণত safer pattern।

45. `const` মানে Array Immutable নয়

এটা খুব common misconception।

```
const fruits = ["Apple", "Banana"];

fruits.push("Mango");
```

এটা perfectly allowed।

কারণ `const` variable-টাকে অন্য array-তে reassign করা আটকায়, array-এর contents পরিবর্তন করা আটকায় না।

```
fruits = ["Orange"];
// ❌ TypeError
```

কিন্তু:

```
fruits.push("Mango");
// ✅
```

46. `const` + Array Mental Model

```
const fruits
  ↓
reference → ["Apple", "Banana"]
```

`const` reference-টা reassign করতে দেয় না:

```
fruits = [];
// ❌
```

কিন্তু referenced array mutate করা যায়:

```
fruits.push("Mango");
// ✅
```

47. `push()` vs Spread

Mutable:

```
const numbers = [1, 2];

numbers.push(3);
```

Immutable-style:

```
const numbers = [1, 2];

const newNumbers = [
  ...numbers,
  3
];
```

React state-এর ক্ষেত্রে দ্বিতীয় pattern useful

48. `unshift()` vs Spread

Mutable:

```
const numbers = [2, 3];

numbers.unshift(1);
```

Immutable-style:

```
const numbers = [2, 3];

const newNumbers = [
  1,
  ...numbers
];
```

49. Remove Last Element Without Mutation

Mutable:

```
const numbers = [1, 2, 3];  
  
numbers.pop();
```

Non-mutating approach:

```
const numbers = [1, 2, 3];  
  
const newNumbers = numbers.slice(0, -1);
```

50. Remove First Element Without Mutation

Mutable:

```
const numbers = [1, 2, 3];  
  
numbers.shift();
```

Non-mutating approach:

```
const newNumbers = numbers.slice(1);
```

Result:

```
[2, 3]
```

51. Insert Without Mutation

ধরো:

```
const numbers = [1, 3];
```

2 insert করতে চাই।

একটি approach:

```
const newNumbers = [  
  ...numbers.slice(0, 1),  
  2,  
  ...numbers.slice(1)  
];  
  
console.log(newNumbers);
```

Result:

```
[1, 2, 3]
```

Practical React code-এ এই ধরনের immutable patterns মাঝে মাঝে দরকার হয়।

52. Replace Without Mutation

ধরো:

```
const fruits = [  
  "Apple",  
  "Banana",  
  "Mango"  
];
```

Index 1 replace করতে:

```
const newFruits = [  
  ...fruits.slice(0, 1),  
  "Orange",  
  ...fruits.slice(2)  
];
```

Result:

```
["Apple", "Orange", "Mango"]
```

তবে বাস্তব application-এ `map()` এই কাজের জন্য আরও readable হতে পারে।

53. `splice()` Empty Array-এ

```
const numbers = [];  
  
const removed = numbers.splice(0, 1);  
  
console.log(removed);  
// []  
  
console.log(numbers);  
// []
```

কোনো element না থাকলে removed array empty।

54. `pop()` Empty Array

```
const numbers = [];  
  
console.log(numbers.pop());  
// undefined
```

55. `shift()` Empty Array

```
const numbers = [];  
  
console.log(numbers.shift());  
// undefined
```

56. `push()` Empty Array

```
const numbers = [];  
  
console.log(numbers.push(10));  
// 1  
  
console.log(numbers);  
// [10]
```

57. `unshift()` Empty Array

```
const numbers = [];  
  
console.log(numbers.unshift(10));  
// 1  
  
console.log(numbers);  
// [10]
```

58. Common Mistake — `push()` Result

ভুল ধারণা:

```
const result = numbers.push(10);  
  
console.log(result);  
// [1, 2, 3, 10]
```

আসলে:

```
result  
// new length
```

Example:

```
const numbers = [1, 2, 3];

const result = numbers.push(10);

console.log(result);

// 4
```

59. Common Mistake — `pop()` Result

```
const numbers = [1, 2, 3];

const result = numbers.pop();

console.log(result);

// 3
```

`result` পুরো array নয়।

60. Common Mistake — `splice()` vs `slice()`

ভুল:

```
const numbers = [1, 2, 3];

numbers.slice(1, 1);
```

এটা delete করবে না।

কারণ `slice()` original array mutate করে না।

Delete করতে:

```
numbers.splice(1, 1);
```

61. Common Mistake — splice() End Index

splice() -এ দ্বিতীয় argument হলো deleteCount, কিন্তু slice() -এ দ্বিতীয় argument হলো end।

```
numbers.splice(1, 2);
```

মানে:

```
index 1 থেকে  
2 elements delete
```

কিন্তু:

```
numbers.slice(1, 2);
```

মানে:

```
index 1 থেকে  
index 2-এর আগ পর্যন্ত copy
```

62. Common Mistake — Direct State Mutation

React:

```
items.push(newItem);  
setItems(items);
```

এটা avoid করা উচিত।

Better:

```
setItems([  
  ...items,  
  newItem  
]);
```

63. Performance Note

`push()` এবং `pop()` সাধারণত array-এর end নিয়ে কাজ করার জন্য efficient। `shift()` এবং `unshift()` -এর ক্ষেত্রে existing elements-এর indexes পরিবর্তন করতে হতে পারে, তাই বড় array-তে এগুলো তুলনামূলক expensive হতে পারে।

তবে normal frontend application-এ performance concern না থাকলে readability এবং correctness আগে।

64. Method Decision Guide

Element শেষে add?

→ `push()`

Element শেষে remove?

→ `pop()`

Element শুরুতে add?

→ `unshift()`

Element শুরু থেকে remove?

→ `shift()`

Anywhere add/remove/replace?

→ `splice()`

Array-এর অংশের নতুন copy?

→ `slice()`

পুরো shallow copy?

→ `slice()`

→ `[...array]`

→ `Array.from(array)`

65. Quick Reference Table

Method	Mutates?	Main Purpose	Return
<code>push()</code>	✓	End-এ add	New length
<code>pop()</code>	✓	End থেকে remove	Removed element
<code>shift()</code>	✓	Start থেকে remove	Removed element
<code>unshift()</code>	✓	Start-এ add	New length
<code>splice()</code>	✓	Add/remove/replace	Removed elements
<code>slice()</code>	✗	Portion copy	New array

66. Practice — Predict the Output

Question 1

```
const arr = [1, 2, 3];

console.log(arr.push(4));
console.log(arr);
```

Question 2

```
const arr = [1, 2, 3];

console.log(arr.pop());
console.log(arr);
```

Question 3

```
const arr = [1, 2, 3];

console.log(arr.shift());
console.log(arr);
```

Question 4

```
const arr = [2, 3];

console.log(arr.unshift(1));
console.log(arr);
```

Question 5

```
const arr = [10, 20, 30, 40];

const removed = arr.splice(1, 2);

console.log(removed);
console.log(arr);
```

Question 6

```
const arr = [10, 20, 30, 40];

const result = arr.slice(1, 3);

console.log(result);
console.log(arr);
```

Question 7

```
const arr = [1, 2, 3];

const copy = arr.slice();

console.log(copy === arr);
```

Question 8

```
const arr = [1, 2, 3];

const copy = arr;

copy.push(4);

console.log(arr);
```

Question 9

```
const arr = ["A", "B", "C"];

arr.splice(1, 0, "X");

console.log(arr);
```

Question 10

```
const arr = ["A", "B", "C"];

arr.splice(1, 1, "X");

console.log(arr);
```

67. Coding Challenges

Challenge 1 — Add Without Mutation

Function:

```
function addItem(array, item) {
  // return a new array
}
```

Expected:

```
const numbers = [1, 2, 3];

const result = addItem(numbers, 4);

console.log(numbers);
// [1, 2, 3]

console.log(result);
// [1, 2, 3, 4]
```

Challenge 2 — Remove Last Without Mutation

```
function removeLast(array) {
  // return a new array without last item
}
```

Expected:

```
removeLast([1, 2, 3]);
// [1, 2]
```

Original array unchanged থাকবে।

Challenge 3 — Remove First Without Mutation

```
function removeFirst(array) {
  // return new array
}
```

Expected:

```
removeFirst([1, 2, 3]);
// [2, 3]
```

Challenge 4 — Insert at Index

```
function insertAt(array, index, value) {  
    // return a new array  
}
```

Expected:

```
insertAt([1, 3], 1, 2);  
// [1, 2, 3]
```

Original array mutate করা যাবে না।

Challenge 5 — Replace at Index

```
function replaceAt(array, index, value) {  
    // return a new array  
}
```

Expected:

```
replaceAt(["A", "B", "C"], 1, "X");  
// ["A", "X", "C"]
```

68. Interview Questions

Beginner

1. `push()` কী করে?
2. `pop()` কী করে?
3. `shift()` কী করে?
4. `unshift()` কী করে?
5. `splice()` কী?
6. `slice()` কী?
7. কোন methods original array mutate করে?
8. `push()` কী return করে?

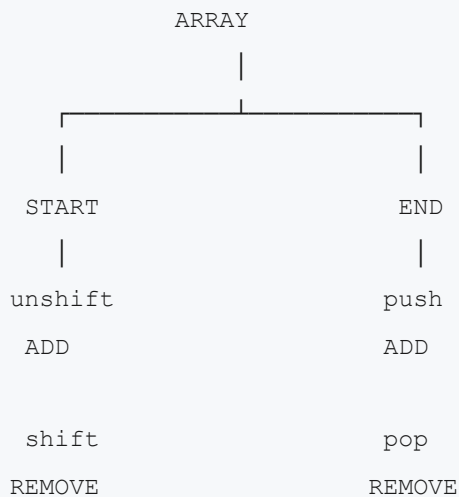
9. `pop()` কী return করে?
10. `splice()` কী return করে?

Intermediate

11. `splice()` এবং `slice()` -এর পার্থক্য কী?
12. `push()` এবং `unshift()` -এর পার্থক্য কী?
13. `pop()` এবং `shift()` -এর পার্থক্য কী?
14. `splice()` দিয়ে কীভাবে insert করবে?
15. `splice()` দিয়ে কীভাবে replace করবে?
16. `slice()` দিয়ে কীভাবে array copy করবে?
17. Shallow copy কী?
18. `const` array-তে `push()` করা যায় কেন?
19. React state array direct mutate করা উচিত নয় কেন?
20. `[...array]` কীভাবে array copy করতে সাহায্য করে?

69. Mental Model

সবগুলো একসাথে মনে রাখার সহজ উপায়:



তারপর:

```
splice()
  → anywhere
  → insert
  → delete
  → replace
  → MUTATES

slice()
  → portion copy
  → DOES NOT MUTATE
```

আর copy:

```
[...array]
array.slice()
Array.from(array)

↓

Shallow Copy
```

70. Golden Rules

1. `push()` → *end-এ add করে এবং new length return করে।*
2. `pop()` → *end থেকে remove করে এবং removed element return করে।*
3. `unshift()` → *start-এ add করে এবং new length return করে।*
4. `shift()` → *start থেকে remove করে এবং removed element return করে।*
5. `splice()` → *original array mutate করে।*
6. `splice()` দিয়ে *add, remove এবং replace—তিনটাই করা যায়।*
7. `splice()` *removed elements-এর একটি array return করে।*
8. `slice()` *original array mutate করে না।*

9. `slice(start, end)` -এ `end` excluded।

10. `splice(start, deleteCount)` -এ দ্বিতীয় *argument* হলো কতটি *element delete* হবে।

11. `slice()` এবং `[...array]` *basic shallow copy* তৈরি করতে পারে।

12. `const` *array immutable* করে না; শুধু *variable reassignment* আটকায়।

13. *React state*-এর *array direct mutate* না করে নতুন *array* তৈরি করে *update* করা ভালো।

14. *Nested objects/arrays* থাকলে *shallow copy*-এর *limitation* মনে রাখতে হবে।